1. Endpoints, métodos y códigos HTTP

Reporte de implementación y evidencia | 12 de noviembre de 2026

Repositorio: e-daga/CDRL-base-2026

Situación inicial. La entrega M09 ejecutaba scripts de Node.js que accedían directamente a PostgreSQL y MongoDB. No tenía servidor HTTP. Para esta actividad se agregó una API local de consulta; los endpoints de este reporte corresponden a esa incorporación.

Funcionamiento: cliente HTTP → servidor Node.js → cuenta MongoDB reader → respuesta JSON. Los datos consultados son fixtures sintéticos existentes. La API no inserta, modifica ni elimina documentos.

Endpoint	Métodos	Qué hace
/health	GET, HEAD	Comprueba conexión a MongoDB.
/api/events	GET, HEAD	Lista paginada por dispositivo.
/api/events/{event_id}	GET, HEAD	Consulta un evento por UUID.
/api/alerts	GET, HEAD	Lista las alertas abiertas.

Métodos. GET obtiene una representación; HEAD devuelve los encabezados correspondientes, sin cuerpo. POST, PUT, PATCH y DELETE no están habilitados y reciben 405 en rutas conocidas. No se requiere implementar todos los métodos HTTP para identificar los que usa esta API.

Código	Significado en esta API
200	Consulta correcta, incluso si la lista está vacía.
400	Parámetros o identificador inválidos.
404	Ruta desconocida o evento inexistente.
405	Método no permitido; encabezado Allow: GET, HEAD.
503 / 500	Dependencia no disponible / fallo interno inesperado.

Fuente de conceptos: RFC 9110, secciones 9.3.1, 9.3.2 y 15. Las capturas siguientes son imágenes de un visor local de respuestas realmente obtenidas por HTTP; no son capturas de Postman. Los fallos inyectados están identificados.

2. Estado del servicio

GET /health ejecuta un ping real usando la cuenta reader. Un 200 indica que la base respondió; no es solamente un texto fijo del servidor. HEAD hace la misma comprobación y omite el cuerpo.



Figura 1. Respuesta real GET: estado 200 y disponibilidad de la base.

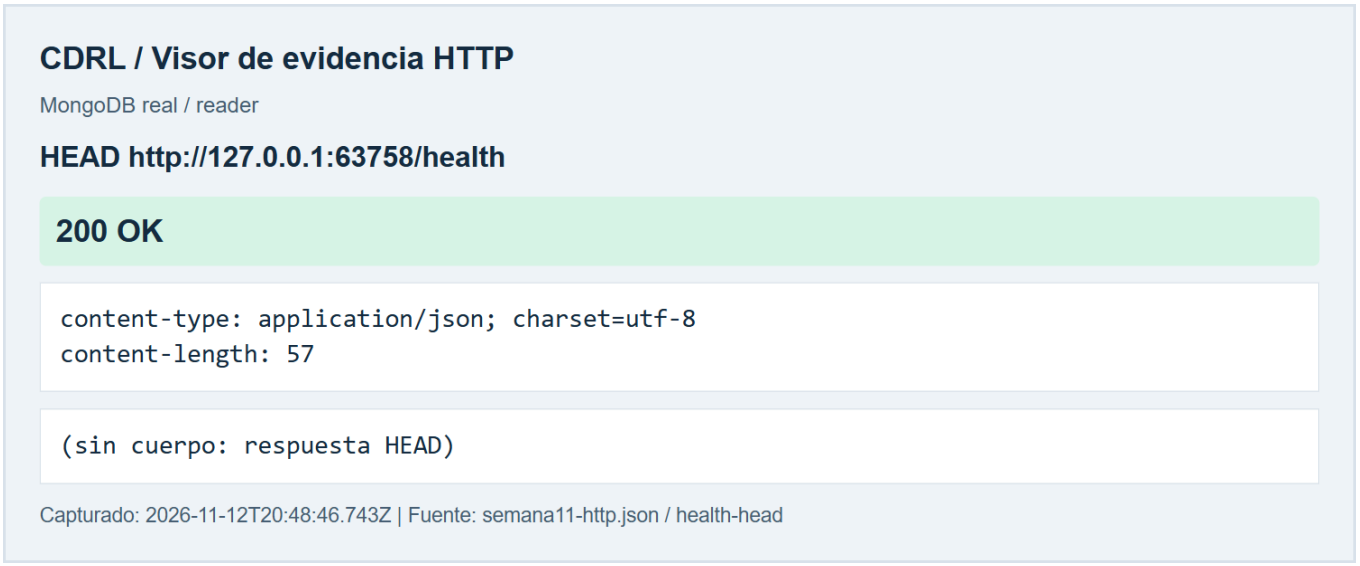


Figura 2. HEAD sobre el mismo endpoint: 200 y cuerpo vacío.

Códigos previstos: 200; 400 si se envían parámetros no admitidos; 405 si se usa otro método; 503 ante un fallo de conexión/configuración; 500 ante otro fallo interno. Los dos últimos se muestran en la sección 6.

3. Consulta paginada de eventos

GET /api/events?deviceId=dev_sensor_01&limit=1. deviceId es obligatorio. limit acepta de 1 a 100; el valor por defecto es 20. from, to y cursor son opcionales. Se reutiliza la paginación de M08.

CDRL / Visor de evidencia HTTP

MongoDB real / reader

GET http://127.0.0.1:63758/api/events?deviceId=dev_sensor_01&limit=1

200 OK

```
content-type: application/json; charset=utf-8
content-length: 392
```


```
{
  "items": [
    {
      "event_id": "a1b2c3d4-0006-4000-8000-000000000006",
      "device_id": "dev_sensor_01",
      "event_type": "battery_pct",
      "observed_at": "2026-09-22T12:00:00.000Z",
      "metric_value": 100,
      "unit": "%",
      "severity": "info"
    }
  ],
  "next_cursor":
"eyJ2IjoxLCJhdCI6IjIwMjYtMDktMjJUMTI6MDA6MDAuMDAwWiIsImlkIjoiYTFiMmMzZDQtMDAwNi00MDAwLTgwMDAtMDAwMDAwMDAwMDA2IiwicSI6IjEyekxWW1CaTBJbEZZYzgifQ",
  "has_more": true,
  "limit": 1
}
```


Capturado: 2026-11-12T20:48:46.493Z | Fuente: semana11-http.json / events

Figura 3. Un evento real, límite aplicado y cursor para continuar la consulta.

CDRL / Visor de evidencia HTTP

MongoDB real / reader

GET http://127.0.0.1:63758/api/events?deviceId=dev_sensor_01&limit=0

400 Bad Request

```
content-type: application/json; charset=utf-8
content-length: 30
```


```
{
  "error": "invalid_parameters"
}
```


Capturado: 2026-11-12T20:48:47.064Z | Fuente: semana11-http.json / invalid

Figura 4. limit=0 es inválido: 400 Bad Request.

HEAD también devuelve 200 sin cuerpo. Una consulta válida sin coincidencias devuelve 200 con items vacío. Posibles errores: 400, 405, 503 y 500. La respuesta selecciona campos y no expone payload ni identificadores internos de MongoDB.

4. Consulta de un evento por ID

GET /api/events/{event_id}. El identificador se obtuvo de la consulta anterior. Un UUID válido que no existe devuelve 404; un identificador mal formado devuelve 400.



Figura 5. Consulta del evento existente: 200 OK.



Figura 6. UUID válido sin documento asociado: 404 Not Found.

HEAD consulta el mismo recurso sin devolver JSON. Otros códigos posibles: 405, 503 y 500. El 404 significa ausencia del recurso, no un error de conexión.

5. Alertas abiertas y método rechazado

GET /api/alerts?limit=1. Filtra status=open. Acepta deviceId, severity y limit; severity puede ser info, warning o critical. HEAD está disponible con los mismos filtros.



Figura 7. Consulta real de alertas abiertas: 200 OK.



Figura 8. POST rechazado: 405 y encabezado Allow con los métodos admitidos.

Posibles códigos: 200, 400, 405, 503 y 500. La API usa una cuenta de solo lectura. El código 13 de MongoDB no se presenta como código HTTP; tampoco se inventan respuestas 201, 401 o 403.

6. Fallos controlados y límites

Estas dos peticiones sí pasaron por HTTP, pero se ejecutaron contra servidores de prueba con una dependencia que lanza un error deliberado. **No se detuvo MongoDB ni se provocó una falla en los datos.**



Figura 9. Fallo MongoDB inyectado: 503 y Retry-After: 5.



Figura 10. Fallo interno inyectado: 500, sin revelar el detalle de la excepción.

Seguridad y alcance. El proceso escucha exclusivamente en 127.0.0.1 y consulta con reader. No tiene autenticación HTTP de usuarios, TLS ni auditoría de peticiones. Es una demostración local para datos sintéticos; no debe exponerse a Internet. Los errores se devuelven con mensajes genéricos y las credenciales permanecen fuera de Git.

Los códigos 500 y 503 acreditan el manejo de errores en una prueba aislada; no prueban una caída de infraestructura real.

7. Comprobación y reproducción

Se registraron **18 intercambios HTTP** con aserciones sobre su código. El JSON de evidencia contiene URL, método, encabezados, cuerpo, fecha UTC y tipo de ejecución. Los puertos de las capturas son temporales; al iniciar normalmente se usa 3000.

Petición comprobada	Resultado
GET /health	200; HEAD 200; POST 405
GET /api/events	200; HEAD 200; POST 405
GET /api/events/{id}	200; HEAD 200; POST 405
GET /api/alerts	200; HEAD 200; POST 405
GET /api/events con limit=0	400
GET /api/events/{UUID ausente}	404
GET /not-a-route	404
GET /api/events sin coincidencias	200, items vacío
GET /health con fallos inyectados	503 y 500

Pruebas automáticas: seis pruebas HTTP nuevas comprueban consultas reales, HEAD, paginación, límites, ausencia, validación, rechazo de escrituras y mensajes de error sin detalles internos. Se descubren también mediante el ejecutor general del repositorio.

Cómo repetirlo:

1. Con Docker funcionando, ejecutar `make setup` si las bases aún no están inicializadas.
2. Ejecutar `npm run serve:http`.
3. Consultar `http://127.0.0.1:3000/health` o usar `curl.exe -i` para ver encabezados y código.
4. Ejecutar `npm run test:http` y `node scripts/capture-http.mjs` para repetir pruebas/evidencia.
5. Ejecutar `make verify` para las regresiones. Detener la API con Ctrl+C.

Trazabilidad del código capturado:
`1f6bbac11d764d0da3c960dac819c743de172a5e`
Rutas: src/http-api.mjs. Arranque: scripts/serve-http.mjs.
Pruebas: tests/http-api.test.mjs. Evidencia: artifacts/semana11-http.json.

Referencia técnica: RFC 9110 - HTTP Semantics, secciones 9.3.1, 9.3.2, 15.3.1, 15.5.1, 15.5.5, 15.5.6, 15.6.1 y 15.6.4. Implementación y evidencia del repositorio son las fuentes de los resultados descritos.

8. Anexo: HEAD en cada consulta

Las tres rutas de datos también fueron consultadas con HEAD. El cuerpo está vacío, aunque Content-Length describe la representación que habría devuelto GET. Estos son los intercambios del mismo registro de evidencia.

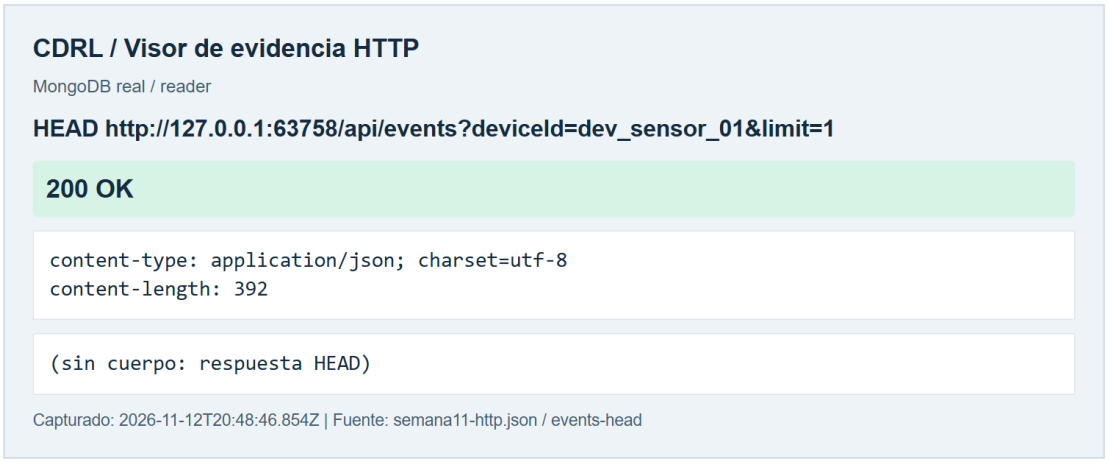


Figura 11. HEAD /api/events: 200.

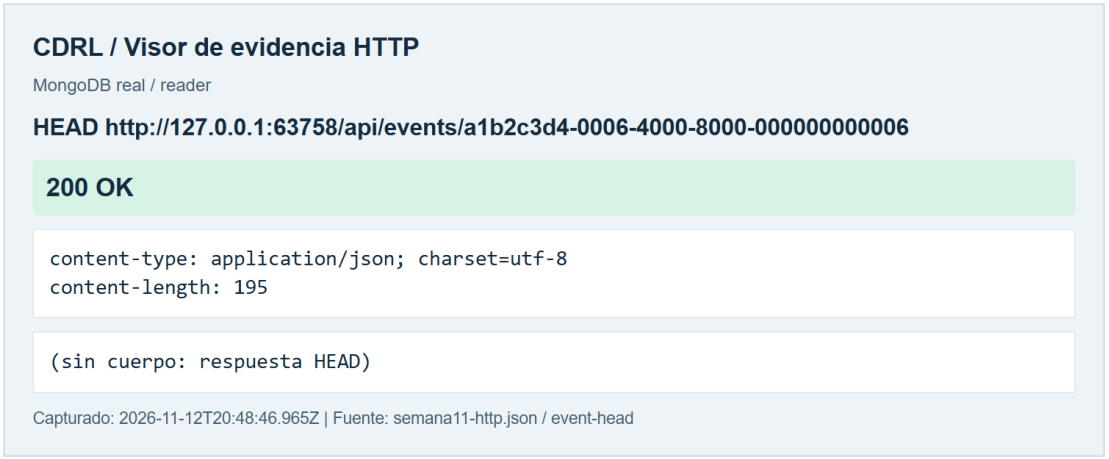


Figura 12. HEAD /api/events/{event_id}: 200.



Figura 13. HEAD /api/alerts: 200.

9. Anexo: rechazo en las demás rutas

Además de POST /api/alerts (figura 8), se comprobó el rechazo de POST en las otras tres rutas. Todas señalan los métodos admitidos mediante Allow. Las pruebas automáticas también cubren PUT, PATCH y DELETE.



Figura 14. POST /health: 405.



Figura 15. POST /api/events: 405.



Figura 16. POST /api/events/{event_id}: 405.

10. Anexo: ausencia frente a lista vacía

Una ruta que no existe devuelve 404. En cambio, una consulta de colección válida sin coincidencias devuelve 200 y una lista vacía. Estos casos no significan lo mismo y se verificaron por separado.

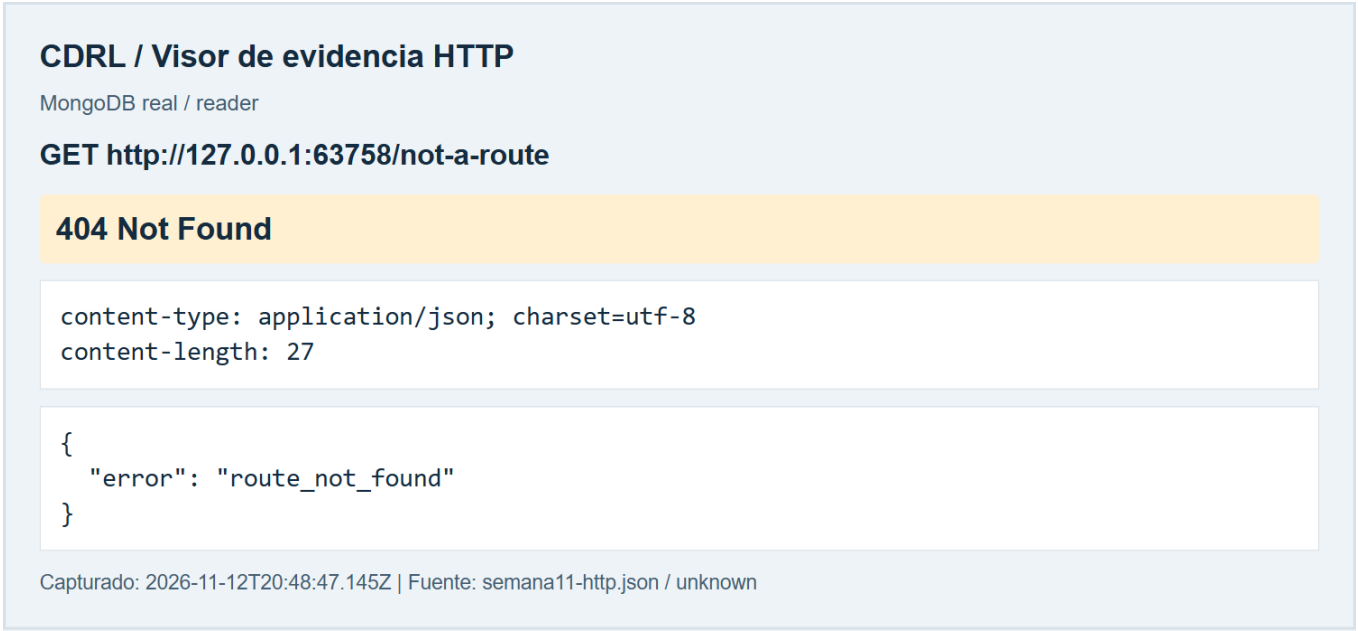


Figura 17. Ruta no registrada: 404 route_not_found.



Figura 18. Dispositivo sin eventos: 200 con items vacío.